Hilmar Daily Shipment Tracker — Enterprise Modernization Plan

Prepared: 2026-05-21 **Owner:** Michael Deitchman (michael.deitchman@idealx.us / @ol-usa.com) **Status:** Proposal — response to the OL enterprise IT / cybersecurity review **Scope:** Infrastructure, authentication, secrets, storage, scheduling, observability, and governance modernization of the Hilmar Daily Shipment Tracker for OL enterprise production ownership.

---

1. Purpose

The OL enterprise IT / cybersecurity review of the Hilmar tracker turnover documentation found the **application and automation logic sound** and the **operational maturity strong**, but flagged the **infrastructure and security architecture** as not yet meeting OL enterprise production standards. This document is the concrete, costed, phased plan to close that gap.

It is deliberately scoped as an **infrastructure and authentication hardening exercise — not an application rewrite**. The parser, QC self-heal framework, 95% accuracy gate, data model, and report generation are not changing (see §11).

This plan is also intended to serve as the **reference pattern** for a repeatable OL enterprise deployment model for future projects — directly answering the review's stated goal.

---

2. Current state (as reviewed)

Layer	Today
Compute	Windows 365 Cloud PC, always-on, RDP-administered
Scheduling	Windows Task Scheduler (`deploy/setup_cloudpc.ps1`)
Auth (mailbox)	Microsoft Graph **delegated**, device-code flow, token cached to `secrets/token-cache.json` on Michael's personal account
Secrets	Local `secrets/` folder — `token-cache.json`, `anthropic-api-key.txt`, `sentry-dsn.txt`, `sentry-auth-token.txt`, `quote-tracker-pwd.txt`
Data store	`tracking-data-v2.json` (155–170 rows) + `data-backups/` on the filesystem / OneDrive
File storage	Local filesystem + OneDrive sync
Observability	Sentry (`idealx-llc` / `hilmar-daily-tracker`) + Seer + Claude fallback
Governance	Single-operator, no centralized RBAC

Already done well (per the review, and confirmed): proper Graph API usage (no Outlook scraping), strong runbooks, the QC / self-heal framework, monitoring concepts, GitHub source control, the dedicated-service-mailbox recommendation, and an Azure deployment direction.

---

3. The core problem: authentication (security priority #1)

Everything else on the list is good hygiene. **This is the one genuine security blocker**, and we are treating it as the #1 item.

Why the current model is not enterprise-grade:

- It uses **delegated** permissions — the pipeline acts *as a signed-in human* (Michael's account), not as a service. Pipeline actions are attributed to a person, not a service principal.
- The credential is a **refresh token cached to disk** (`token-cache.json`). Any process or person with file/RDP/OneDrive access to that file holds a bearer credential.
- The token is invalidated by routine events — password change, MFA re-registration, Conditional Access policy change — and recovery requires a **human to interactively re-authenticate**. That is the "manual token refresh" the review flagged.
- It creates an **offboarding** / **continuity risk**: the production pipeline is coupled to one individual's identity lifecycle.

The fix (target design):

1. A dedicated **Entra app registration** — e.g. `OL-Hilmar-Tracker-Svc`.
2. **Application permissions** (app-only, *not* delegated): `Mail.Read`, `Mail.Send`. Requires a **one-time OL tenant admin consent**.
3. An **Application Access Policy** (Exchange Online) restricting the app to **a single mail-enabled group containing only the Hilmar service mailbox**. *This is the most important security control in the whole design:* without it, `Mail.Read` / `Mail.Send` application permissions grant tenant-wide mailbox access. With it, the app can only ever touch the one Hilmar mailbox — true least privilege.
4. **Credential — preferred: workload identity federation.** The Container App's managed identity federates to the app registration as a federated credential. Result: **no certificate or secret stored or rotated anywhere.** *Fallback:* a certificate credential with the private key in Azure Key Vault (never on disk).
5. The pipeline runs as a **service principal** — fully unattended, no device-code, no manual refresh, no personal-account dependency, clean audit trail.

Code impact is contained — `scripts/outlook_send.py` swaps the MSAL `PublicClientApplication` + device-code path for a confidential-client app-only flow (or `azure-identity` `DefaultAzureCredential`), and Graph calls move from `/me/...` to `/users/{service-mailbox}/...`. The daily email also moves to sending **as the dedicated service mailbox** rather than a personal `@ol-usa.com` address.

---

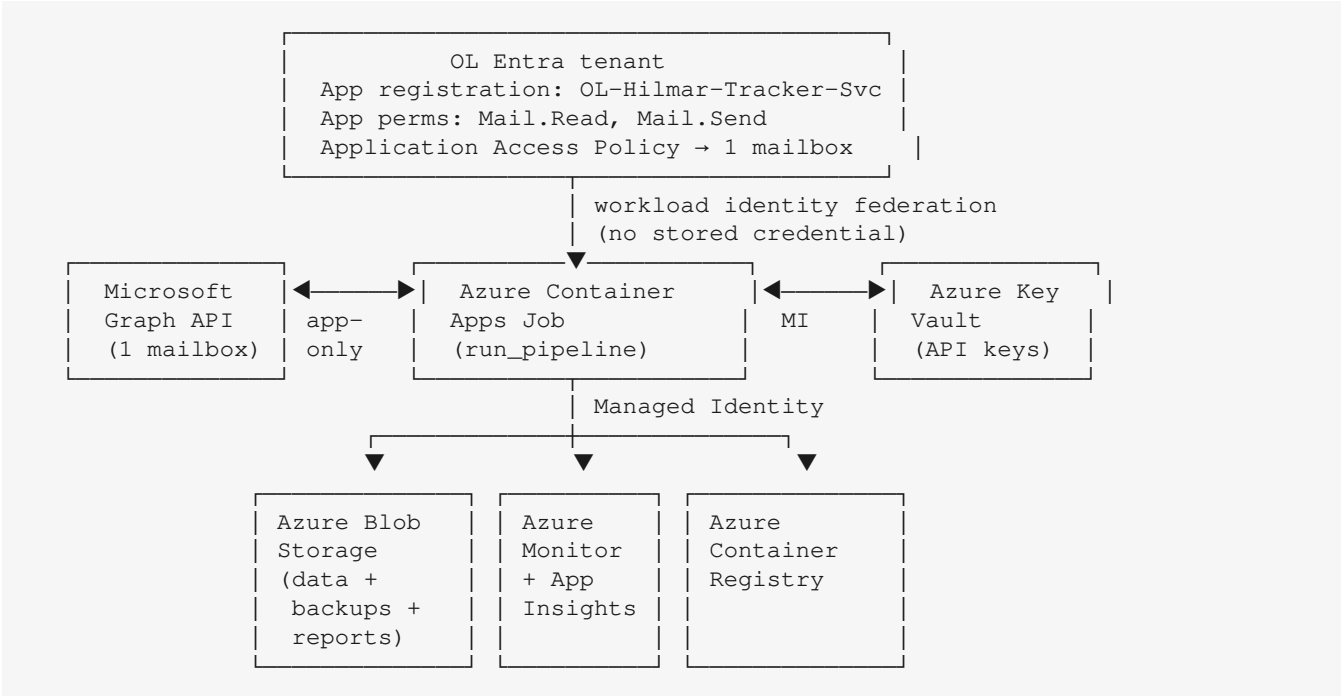
4. Current → target mapping

The review's recommended replacements are largely correct. Two are adjusted below for cost-efficiency, with reasons — these are the only points where we respectfully diverge.

Layer	Current	Target	Notes
Compute	Windows VM / Cloud PC	**Azure Container Apps Job**	Consumption-billed; runs ~15 min/day
Scheduling	Windows Task Scheduler	**Container Apps Job cron** (or Logic App for exact-ET)	See §7 DST note
Mailbox auth	`token-cache.json` device-code	**App-only auth + Application Access Policy**	§3 — the security fix

Azure-resource auth	n/a	**Managed Identity**	Service-to-service, no secrets
Secrets	Local `secrets/` folder	**Azure Key Vault**	§6
Data store	JSON file	**Azure Blob Storage (versioned)** — *not Postgres/SQL*	§8 — adjusted, see reasoning
File storage	Local FS / OneDrive	**Azure Blob Storage**	Reports, PDFs, backups
Observability	Sentry	**Azure Monitor + Application Insights** (Datadog optional)	§9 — see honest comparison
Governance	Single-operator	**Centralized RBAC + Entra groups**	§10

5. Target architecture



Scheduled trigger (cron or Logic App) starts the Container Apps Job each weekday; the job authenticates via managed identity, pulls secrets from Key Vault, reads/writes the data file in Blob Storage, calls Graph app-only against the single Hilmar mailbox, and ships telemetry to Azure Monitor / Application Insights.

6. Secrets — Azure Key Vault

- One Key Vault, e.g. `kv-hilmar-tracker`.
- Holds: Anthropic API key, the observability tool's API/ingestion key, any DB connection string, and the Graph cert **only if** federation is not used.
- The Container App's **managed identity** is granted `Key Vault Secrets User`; secrets are surfaced as Container Apps secret references or read via the SDK at runtime.
- The on-disk `secrets/` folder is **eliminated**. `token-cache.json` is retired entirely (no token cache in the app-only model).
- Rotation, expiry alerts, and access logging become Key Vault native functions instead of manual file management.

7. Compute & scheduling

- **Containerize:** a `Dockerfile` on a `python:3.11-slim` base. Note WeasyPrint / PDF rendering needs system libraries (`libpango`, `libcairo`, fonts) installed in the image.
- Image pushed to **Azure Container Registry** (Basic tier).
- **Azure Container Apps Job** (Schedule trigger type) replaces Windows Task Scheduler — consumption-billed, runs only for the ~15-minute daily window.
- **DST caveat (honest):** Container Apps Job cron expressions are **UTC only**. 10:00 AM ET is 14:00 UTC under EDT and 15:00 UTC under EST. Three options: (a) accept a one-hour seasonal drift, (b) maintain two seasonal cron entries, or (c) trigger the job from an **Azure Logic App** with a timezone-aware recurrence (recommended if exact 10:00 ET matters). The Logic App option is the cleanest "Azure-native scheduling" answer.
- The **Windows 365 Cloud PC dependency is decommissioned** after parallel validation — no always-on machine, no RDP administration.

---

8. Data storage — recommended adjustment

The review recommends Azure PostgreSQL or Azure SQL. We recommend Azure Blob Storage instead, and here is the reasoning.

`tracking-data-v2.json` is a **155–170 row** dataset — one JSON document. A managed relational database for a dataset this size adds material monthly cost and operational surface (patching, connection management, schema migrations) for **no functional benefit at this scale**.

The legitimate concern behind "move off a flat file" is real — a local file has no versioning, no audit trail, no access control, and no managed backup. **Azure Blob Storage with blob versioning and soft-delete enabled solves exactly those concerns:**

- **Versioning** → point-in-time history and an audit trail of every change.
- **Soft-delete** → recovery from accidental deletion.
- **Lifecycle management** → automated backup retention (replaces the manual `data-backups/` pruning).
- **RBAC + managed identity** → governed, least-privilege access.
- Encryption at rest and in transit by default.

Recommendation: Blob Storage container `hilmar-tracker-data` for the data file, backups, and generated reports/PDFs.

Concession path: if OL governance mandates a relational store as a hard standard, use **Azure SQL Database (serverless tier)** — it auto-pauses to near-zero cost when idle, which suits a once-daily workload far better than provisioned PostgreSQL. We would still note this solves a problem this dataset does not have, but it is a defensible standard-compliance choice.

---

9. Observability — recommended: Azure Monitor + Application Insights

The review recommends moving off Sentry; Datadog was raised as a possible alternative. **Is Datadog the better choice here? For this specific workload — no.** Honest assessment of the three options:

This pipeline is a **single small batch job** — one Python process, ~15 minutes a day, a ~170-row dataset. Its monitoring need is narrow: *did the daily run complete, and did it throw exceptions.* Against that need:

- **Azure Monitor + Application Insights — recommended.** The only option that is genuinely **Azure-native**: telemetry stays inside the OL tenant, it is billed on the Azure invoice, and it is governed by the same Entra RBAC as everything else. This is the option that actually satisfies the "keep it in our Azure environment" governance driver behind the review. App Insights covers exception tracking, the cron heartbeat (availability test / "no-data" alert), and the custom metrics. Its error-grouping UX is weaker than Sentry's and it has no built-in AI autofix — acceptable for one batch job.
- **Datadog — not recommended for this workload.** Datadog is a best-in- class platform for **broad infrastructure / APM observability across a fleet of services**. A single daily cron job exercises almost none of that, and Datadog is the **most expensive** of the three. Important nuance: Datadog's Azure integration means it is *billed through* Azure, but telemetry **still egresses to Datadog's SaaS** — so it is **not** "Azure-native" in the data-residency sense, and does not satisfy the governance driver any better than Sentry already does. Datadog becomes the right answer only if OL standardizes on it across many projects.
- **Keeping Sentry — defensible.** Sentry is purpose-built for application error tracking — the exact need here — and already carries the closed-loop self-heal. The review's "replace Sentry" is a standardization *preference*, not a security finding. If OL has no mandated observability standard, keeping Sentry is the lowest-cost, lowest-risk path. It is a third-party SaaS — but so is Datadog.

Recommendation: if a move off Sentry is required for OL standardization, target **Azure Monitor + Application Insights** — the genuinely Azure-native, tenant-resident, governance-aligned, and cheaper choice. Adopt Datadog only if it is (or becomes) the OL-wide observability standard.

Migration note: moving off Sentry **loses the current closed-loop self-heal** (`scripts/qc_actions_from_sentry.py` — Seer autofix + Claude diagnosis) — the largest code item after the auth change. The good news: the **Claude-diagnose fallback is vendor-agnostic** and ports directly; only the issue-polling layer re-points from the Sentry API to the target's API. `scripts/sentry_setup.py`, `sentry_api.py`, and `sentry_seer.py` are replaced by Application Insights SDK init (OpenTelemetry); the custom metrics (parser accuracy, pipeline duration, QC counts, send health) become App Insights custom metrics; the Sentry cron heartbeat becomes an App Insights availability / "no-data" alert.

---

10. Governance, RBAC & ownership

- **Resource groups:** `rg-hilmar-tracker-sandbox` and `rg-hilmar-tracker-prod`, each tagged per OL tagging policy.
- **Access via Entra groups, never individuals** — e.g. an owners group (platform team), a contributors group, a readers group (stakeholders).
- **Managed identity** for all service-to-service auth; **no shared credentials**.
- **Key Vault** access is RBAC-scoped; secret access is logged.
- **Diagnostic settings** on every resource → Log Analytics / Azure Monitor.
- **Azure Policy** enforces tagging, allowed regions, and resource SKUs.
- **Ownership matrix** — to be agreed with OL IT: who owns the application, who owns the infrastructure, and the support / on-call path.

---

11. What does NOT change (reassurance / scope control)

The following are **explicitly out of scope** — they are working, tested, and reviewed as sound:

- The parser and the **95% accuracy hard gate** (`parser_accuracy.py`).

- The **QC / self-heal framework** (QC-001..QC-051).
- The status state machine and data model.
- The daily email, HTML dashboard, and PDF generation.
- The test suite and coverage gate.

This keeps the modernization low-risk: it is an infrastructure and authentication migration around an unchanged, validated application core.

---

12. Phased roadmap

Phase	Work	Depends on	Rough effort
0 — Interim hardening	Request the dedicated service mailbox + app registration from OL tenant admin; tighten `secrets/` folder ACLs; confirm `.gitignore` coverage; rotate long-lived credentials; document the RBAC/ownership model	OL tenant admin engagement	Days
1 — Auth modernization	App-only Graph auth, Application Access Policy scoping, workload identity federation (or cert in Key Vault); rework `outlook_send.py`	OL admin consent + Access Policy	1–2 weeks
2 — Secrets + storage	Key Vault; Blob Storage (versioned) for data, backups, reports; managed-identity access	Phase 1	1 week
3 — Compute + scheduling	Dockerfile, ACR, Container Apps Job, scheduled trigger; decommission Cloud PC	Phases 1–2	1–2 weeks
4 — Observability	Azure Monitor + Application Insights re-instrumentation; port the closed-loop self-heal	Phase 3	1–2 weeks
5 — Governance + cutover	RBAC, Azure Policy, ownership sign-off, runbook update, **sandbox parallel run**, then **production cutover last**	Phases 1–4	1 week + parallel-run window

Sequencing matches the review's guidance: sandbox first, parallel-run validation against the current Cloud PC, production cutover last. **The long-pole dependency is the OL tenant admin** (app registration, admin consent, Application Access Policy) — Phase 0 starts that conversation immediately so it does not block Phase 1.

---

13. Indicative cost estimate

Monthly, USD, **estimates** — final numbers depend on OL's Enterprise Agreement rates and any existing observability-tool contract.

Service	Estimate / month	Notes
Container Apps Job	$0–10	Consumption; ~5 vCPU-hours/month, largely within the free grant
Azure Container Registry (Basic)	~$5	
Key Vault	~$1–3	Operation-based
Blob Storage (versioned)	~$1–5	A few MB of JSON + backups + reports

Azure Monitor + App Insights	~$5–15	Ingestion-based; small telemetry volume
Logic App (if used for exact-ET schedule)	~$0–5	Consumption
Total (recommended design)	**~$15–45/mo**	
Datadog instead of Azure Monitor	*+$20–60*	*Not recommended for a single batch workload (§9)*
Add Azure SQL serverless (concession path)	*+$15–40*	*Only if a relational store is mandated*
Provisioned PostgreSQL instead	*+$50–150*	*Not recommended at this data scale*

Headline for the budget conversation: a properly governed Azure deployment of *this* workload is on the order of **$15–45/month** with the recommended design. The cost is modest and mostly fixed. The two optional upgrades the review raised — a relational database and Datadog — together could more than triple that figure for no functional gain at this scale, which is why §8 and §9 recommend against them.

---

14. Risks & dependencies

Risk / dependency	Mitigation
OL tenant admin availability (app registration, admin consent, Access Policy)	Start Phase 0 immediately; it is the critical path
Application Access Policy mis-scoped → over-broad mailbox access	Treat the Access Policy as a security gate; verify with `Test-ApplicationAccessPolicy` before go-live
Loss of Sentry self-heal loop during the observability migration	Keep the vendor-agnostic Claude-diagnose fallback; run the old and new observability stacks in parallel through the migration
DST drift on UTC-only cron	Use the Logic App timezone-aware trigger
PDF rendering dependencies in the container	Bake WeasyPrint system libraries into the image; validate in sandbox
Cutover regression	Sandbox parallel-run against the live Cloud PC before production cutover

---

15. Decision matrix — pros, cons, security, governance, cost

Each major modernization decision is spelled out below across the five lenses the review cares about: **pros**, **cons** / **tradeoffs**, **security**, **enterprise & governance**, and **cost**. Costs are monthly USD estimates (see §13 for the consolidated total).

15.1 Compute — Windows Cloud PC → Azure Container Apps Job

- **Pros:** no always-on machine to maintain; consumption billing (runs only the ~15-minute daily window); immutable, version-controlled container image; no desktop-OS patching burden.
- **Cons** / **tradeoffs:** requires containerizing the app (Dockerfile + PDF system libraries); introduces a build/release pipeline; the team learns Container Apps operations.
- **Security:** eliminates the interactive **RDP attack surface** and the logged-in desktop session; the image is registry-scanned; the job runs as a **managed identity**, not as a signed-in user.
- **Enterprise & governance:** the resource sits in a governed resource group under Azure RBAC and Azure Policy; fully reproducible from source; matches the standard OL deployment pattern.

- **Cost:** ~$0–10 compute + ~$5 container registry.

15.2 Authentication — device-code / token-cache → app-only federated auth

- **Pros:** fully unattended — no device-code prompts, no manual token refreshes; not tied to any individual's account; clean service-principal audit trail; survives password/MFA/Conditional-Access changes.
- **Cons / tradeoffs:** requires a one-time OL tenant-admin action (app registration, admin consent, Application Access Policy); a contained code change in `scripts/outlook_send.py`.
- **Security — this is the #1 item.** Removes the on-disk bearer credential (`token-cache.json`); removes the personal-account and offboarding risk; the **Application Access Policy** scopes the app to exactly **one mailbox**, so even the app's permissions cannot reach other mailboxes; with workload identity federation there is **no stored secret to leak or rotate at all**.
- **Enterprise & governance:** the pipeline runs as a governed service principal with least-privilege, admin-consented permissions — the enterprise-grade unattended-auth pattern the review asked for.
- **Cost:** ~$0 with workload identity federation; certificate fallback adds only its Key Vault storage (counted in §15.3).

15.3 Secrets — local `secrets/` folder → Azure Key Vault

- **Pros:** centralized, encrypted, access-logged secret storage; rotation and expiry alerting are native; the on-disk `secrets/` folder is eliminated.
- **Cons / tradeoffs:** application reads secrets over the network at startup (negligible for a daily job); one more resource to provision.
- **Security:** no plaintext credentials on any filesystem or in OneDrive sync; access is gated by managed identity and logged for audit; blast radius of a host compromise drops sharply.
- **Enterprise & governance:** RBAC-scoped access, full audit trail, and centralized rotation policy — a hard OL requirement.
- **Cost:** ~$1–3 (operation-based).

15.4 Scheduling — Windows Task Scheduler → Azure-native trigger

- **Pros:** scheduling is declarative and version-controlled; no dependency on a specific machine being awake; restart/retry handled by the platform.
- **Cons / tradeoffs:** Container Apps Job cron is **UTC-only** — exact 10:00 ET across DST needs either two seasonal entries or a timezone-aware Logic App trigger.
- **Security:** no scheduled task defined on an interactive desktop; trigger configuration is governed as code.
- **Enterprise & governance:** the schedule is an Azure resource under RBAC, visible and auditable centrally rather than buried in a Windows desktop.
- **Cost:** ~$0 (Container Apps cron) or ~$0–5 (Logic App, if used for exact-ET scheduling).

15.5 Data store — JSON file → Azure Blob Storage *(vs the review's relational DB)*

- **Pros (Blob, recommended):** versioning + soft-delete give change history, an audit trail, and point-in-time recovery; lifecycle policy automates backup retention; minimal cost; no DB to patch or tune.
- **Cons / tradeoffs (Blob):** not a queryable relational store — acceptable here because the whole dataset is one 155–170 row JSON document loaded into memory.
- **Pros / cons of the relational alternative:** PostgreSQL / Azure SQL add query, constraints, and concurrency the workload does not need, at materially higher cost and operational overhead. If a relational store is a hard OL standard, **Azure SQL serverless** (auto-pause) is the lighter, cheaper

choice over provisioned PostgreSQL.
- **Security:** either option gives encryption at rest/in transit and RBAC; Blob with managed-identity access removes any connection string entirely.
- **Enterprise & governance:** Blob versioning satisfies the governance intent behind "move off a flat file" — audit trail, recovery, governed access — without the DB footprint.
- **Cost:** Blob ~$1–5; Azure SQL serverless +$15–40; provisioned PostgreSQL +$50–150 (**not recommended at this data scale**).

15.6 Observability — Sentry → Azure Monitor + App Insights *(vs Datadog)*

- **Pros (Azure Monitor + App Insights, recommended):** genuinely Azure-native — telemetry stays inside the OL tenant; billed on the Azure invoice; governed by the same Entra RBAC; covers exception tracking, the cron heartbeat, and custom metrics.
- **Cons / tradeoffs:** weaker error-grouping UX than Sentry; no built-in AI autofix; the closed-loop self-heal must be re-instrumented.
- **Pros / cons — Datadog:** best-in-class for fleet-scale infra/APM, but **overkill and the most expensive option** for one daily batch job; and although billed through Azure, telemetry **still egresses to a third-party SaaS**, so it is not Azure-native for data-residency purposes.
- **Pros / cons — keeping Sentry:** purpose-built for error tracking and already carrying the self-heal loop; lowest cost and lowest migration risk; but it is a third-party SaaS and not the OL standard.
- **Security:** Azure Monitor keeps all telemetry tenant-resident — the strongest data-residency posture of the three.
- **Enterprise & governance:** Azure Monitor is the only option that is natively governed by OL Azure RBAC and policy; Datadog only if OL standardizes on it org-wide.
- **Cost:** Azure Monitor + App Insights ~$5–15; Datadog +$20–60; keeping Sentry adds no Azure cost (existing Sentry subscription only).

15.7 Governance & RBAC — single-operator → centralized model

- **Pros:** access via Entra groups, not individuals; clear ownership matrix; Azure Policy enforces tagging, region, and SKU compliance.
- **Cons / tradeoffs:** requires up-front agreement with OL IT on roles and the ownership/on-call matrix.
- **Security:** least-privilege role assignments; no shared credentials; service-to-service auth via managed identity; centralized access logging.
- **Enterprise & governance:** this *is* the governance layer — it is what makes the system OL-ownable, and it becomes the reusable reference pattern for future projects.
- **Cost:** ~$0 incremental — RBAC and Azure Policy are built-in Azure features.

---

16. Summary

The review's central finding is correct and we agree with it: the **authentication model is the real security gap**, and the supporting infrastructure should be centralized and governed. This plan resolves the authentication risk with **app-only, federated, least-privilege auth**, moves secrets to **Key Vault**, compute to **Container Apps**, scheduling to **Azure-native triggers**, storage to **Blob Storage**, and observability to **Azure Monitor + Application Insights**.

We diverge on two cost-driven points only: **Blob Storage over a relational database** for a 170-row dataset, and **Azure Monitor + Application Insights over Datadog** — Datadog is a fleet-scale platform and is overkill (and the priciest option) for a single daily batch job (§9).

The application core is not changing. Delivered as the phased plan above, this becomes OL's **repeatable enterprise deployment reference pattern** for future projects — exactly the outcome the review asked for.